

LexiRead — 个人阅读记录与词汇积累工具

产品需求文档 · Product Requirements Document

文档版本：v1.0

创建日期：2026-05-26

适用开发者：独立开发者 / AI 辅助开发

技术栈：React + Vite + Tailwind CSS

模块一：产品概览 · Product Overview

一句话定位

LexiRead 是一款极简的**双语阅读伴侣**——帮助正在阅读英语/法语书籍的用户，在同一个地方完成「记录进度 + 查询生词 + 积累词汇」三件事，从而养成可持续的阅读习惯。

💡 核心洞察（来自用户访谈）

用户痛点	LexiRead 的解法
遇到生词就切换 app，很麻烦	内置 AI 划词查询，查完直接存入词库
查了就忘，没有留存	每个生词自动归档到「我的词库」
没有阅读仪式感，难以坚持	书籍进度条 + 词汇积累数字，让成长可视化
读的是 Kindle / PDF / epub，工具不连通	定位为「阅读后的整理工具」，粘贴文段即可使用

👤 目标用户

正在自学英语/法语、阅读原版书籍的成年人。没有技术背景，需要极低的使用门槛。

产品边界 (MVP 范围)

做:

- 管理书单 + 记录阅读进度
- 粘贴文段、划词查询 (AI 驱动)
- 生词自动保存到词库
- 每本书的词汇归档

不做 (v1 不涉及):

- 社交分享
 - 书籍信息自动抓取 (ISBN 搜索)
 - 用户账号系统
 - 复习提醒 / 间隔重复
-

模块二: 核心功能 · Core Features

Feature 1 · 我的书架

- 用户可以手动添加一本书 (书名 + 语言 + 总页数)
- 每本书显示: 封面色块 (无图)、书名、语言标签 (EN / FR)、进度条 (已读页 / 总页数)
- 点击书本进入「书本详情页」

Feature 2 · 阅读进度更新

- 在书本详情页, 用户可以输入「今天读到第 X 页」
- 进度条实时更新, 显示百分比
- 显示「距离读完还有 X 页」

Feature 3 · 文段查词器

- 用户可以在书本详情页粘贴一段原文 (英/法文)

- 在文段中长按或划选某个单词
- 弹出卡片显示：
 - 单词本身
 - AI 生成的释义（中文 + 例句）
 - 「保存到词库」按钮
- 用户可补充自己的备注

Feature 4 · 我的词库

- 全局词库：展示所有已保存的单词
- 可按书本筛选
- 每条词条显示：单词、释义、来源书目、保存日期
- 支持搜索

Feature 5 · 积累仪表盘

- 首页顶部显示：
 - 已读完书籍数
 - 累计认识单词数
 - 正在读的书（快捷入口）

模块三：技术框架 · Tech Stack

```
前端框架:    React 18 + Vite
样式方案:    Tailwind CSS v3
AI 查词:     DeepSeek API
数据存储:    localStorage (MVP 阶段, 无需后端)
路由:        React Router v6
状态管理:    React Context + useReducer
部署:        Vercel / Netlify (静态托管)
```

AI 查词调用逻辑

```
// 用户划选单词后，调用 DeepSeek API 获取释义
const prompt = `
  你是一位语言教师。请为以下单词提供简洁的中文释义和一个例句。
  单词: ${selectedWord}
  来源语言: ${book.language}  // "EN" 或 "FR"

  请用 JSON 格式返回：
  {
    "word": "单词",
    "definition": "中文释义",
    "example": "原文例句",
    "example_translation": "例句中文翻译"
  }
`;
```

模块四：UI 设计风格 · Design System

🧠 设计理念

「纸张质感的数字书房」——温暖、克制、有书卷气。让每次打开 app 都像翻开一本喜欢的书。

色板

```
:root {
  /* 主色调：暖米白 */
  --color-bg: #FAF7F2; /* 页面背景，类羊皮纸 */
  --color-surface: #FFFFFF; /* 卡片背景 */
  --color-border: #E8E0D5; /* 边框，柔和 */

  /* 强调色：深墨绿 */
  --color-accent: #2D5016; /* 主按钮、进度条 */
  --color-accent-light: #EBF3E1; /* 标签背景 */

  /* 文字 */
  --color-text-primary: #1A1A1A; /* 正文 */
  --color-text-secondary: #6B6560; /* 副文本 */
  --color-text-muted: #A89F97; /* 提示文字 */

  /* 功能色 */
  --color-en: #3B5BDB; /* 英语标签：深蓝 */
  --color-fr: #C92A2A; /* 法语标签：深红 */
}
```

```
}

```

字体

```
font-family: 'Lora', Georgia, serif;          /* 书名、标题——有书卷气的
衬线体 */
font-family: 'Inter', system-ui, sans-serif; /* 正文、按钮——清晰易读
*/

```

视觉规范

属性	规格
圆角	rounded-xl (卡片) / rounded-full (标签、按钮)
阴影	shadow-sm, 只用于卡片悬停态, 避免厚重感
间距基准	8px 系统 (p-2, p-4, p-6, p-8)
动效	transition-all duration-200 ease-in-out, 克制
图标	Lucide React (线条图标, 轻盈)

模块五： 页面结构 · Page Structure

```

/  ───────────  首页 · Dashboard
|  ──  积累仪表盘 (已读书数 / 词汇量)
|  ──  「正在读」快捷卡片
|  ──  「+ 添加新书」按钮

/books ───────────  书架页
|  ──  BookCard × n (进度条 + 语言标签)

/books/:id ───────────  书本详情页  ★ 核心页面
|  ──  书名 + 语言 + 进度更新
|  ──  文段查词器 (TextReader 组件)

```

```
| | | WordPopup (划词弹出卡片)
| | └─ 本书词库 (快速预览)

/vocab ─────────── 全局词库页
| | | 搜索框
| | | 按书筛选
| | └─ VocabCard × n
```

组件拆分

```
components/
├─ layout/
|   ├── NavBar.jsx           # 底部导航 (移动端) / 侧边导航 (桌面端)
|   └─ PageWrapper.jsx      # 统一页面容器
├─ books/
|   ├── BookCard.jsx         # 书架上的书本卡片
|   ├── AddBookModal.jsx     # 添加书本弹窗
|   └─ ProgressUpdater.jsx   # 进度更新组件
├─ reader/
|   ├── TextReader.jsx       # 文段粘贴 + 划词区域
|   └─ WordPopup.jsx         # 划词后弹出的释义卡片
└─ vocab/
    ├── VocabCard.jsx        # 单个词条卡片
    └─ VocabFilter.jsx       # 筛选 / 搜索栏
```

模块六：项目结构 · Project Structure

```
lexiread/
├─ public/
|   └─ favicon.ico
├─ src/
|   ├── main.jsx             # 入口文件
|   ├── App.jsx              # 路由配置
|   ├── index.css            # Tailwind 指令 + CSS 变量
|   ├── context/
|   |   └─ AppContext.jsx    # 全局状态 (书单 + 词库)
|   ├── hooks/
|   |   ├── useBooks.js      # 书本 CRUD 逻辑
|   |   ├── useVocab.js      # 词库 CRUD 逻辑
|   |   └─ useWordLookup.js  # AI 查词逻辑
```

```
├── pages/
│   ├── Dashboard.jsx
│   ├── Bookshelf.jsx
│   ├── BookDetail.jsx
│   └── VocabLibrary.jsx
├── components/           # 见模块五
├── services/
│   └── claudeApi.js       # Anthropic API 封装
├── utils/
│   └── storage.js         # localStorage 读写封装
├── .env                   # VITE_ANTHROPIC_API_KEY=...
├── vite.config.js
├── tailwind.config.js
└── package.json
```

模块七：数据模型 · Data Model

所有数据存储于 `localStorage`, key 为 `lexiread_books` 和 `lexiread_vocab`。

Book (书本)

```
interface Book {
  id: string;           // uuid, e.g. "b_1a2b3c"
  title: string;        // 书名, e.g. "Le Petit Prince"
  author?: string;      // 作者 (可选)
  language: "EN" | "FR"; // 语言
  totalPages: number;   // 总页数
  currentPage: number;  // 当前读到第几页
  status: "reading" | "finished" | "paused";
  coverColor: string;   // 随机生成的封面色, e.g. "#D4A373"
  createdAt: string;    // ISO 日期
  updatedAt: string;
}
```

VocabEntry (词条)

```
interface VocabEntry {
```

```
id: string;           // uuid, e.g. "v_4d5e6f"
word: string;         // 原始单词, e.g. "éphémère"
language: "EN" | "FR"; // 单词语言
definition: string;   // AI 生成的中文释义
example: string;      // AI 生成的例句 (原文)
exampleTranslation: string; // 例句中文翻译
userNote?: string;    // 用户自己补充的备注
bookId: string;       // 来源书本 ID
bookTitle: string;    // 来源书名 (冗余存储, 方便展示)
savedAt: string;      // ISO 日期
}
```

AppStats (首页仪表盘用, 动态计算, 不存储)

```
interface AppStats {
  totalBooksFinished: number; // books.filter(b => b.status ===
    "finished").length
  totalWordsLearned: number; // vocab.length
  currentlyReading: Book[]; // books.filter(b => b.status ===
    "reading")
}
```

附录：开发优先级 · Build Order

建议按此顺序开发，每完成一步就有可用的产品。

Phase 1 · 骨架 (1-2 天)

- ☒ 项目初始化 (Vite + React + Tailwind)
- ☒ 路由搭建 (4 个页面)
- ☒ localStorage 工具函数

Phase 2 · 书架 (2-3 天)

- ☒ 添加书本
- ☒ 书架展示
- ☒ 进度更新

Phase 3 · 查词核心 (3-4 天)

- ☒ 文段粘贴区
- ☒ 划词选择逻辑
- ☒ Claude API 接入

☒ WordPopup 弹窗

☒ 保存到词库

Phase 4 · 词库 + 收尾 (1-2 天)

☒ 词库页面

☒ 搜索 + 筛选

☒ 首页仪表盘

☒ 响应式适配 (移动端)